GOSIM

GOSIM Paris 2026

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris

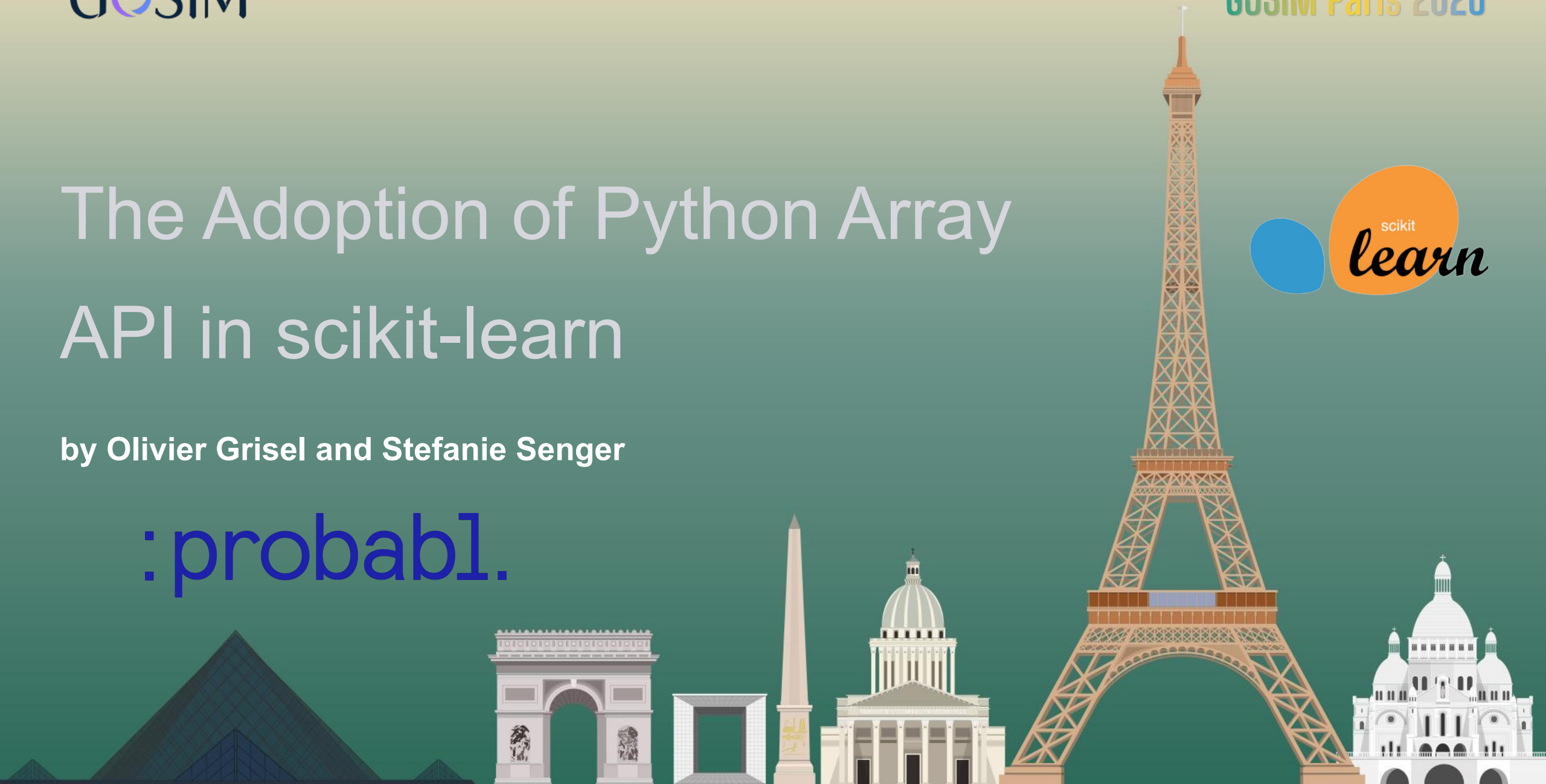
GOSIM

GOSIM Paris 2026

The Adoption of Python Array API in scikit-learn

by Olivier Grisel and Stefanie Senger

:probabl.



CONTENTS

GOSIM Paris 2026

Part 01 Adopting the Array API Standard in scikit-learn

Part 02 Implementation and Challenges

Part 03 单击此处修改标题

Part 04 单击此处修改标题

GOSIM

GOSIM Paris 2026

01

Adopting the Array API Standard in scikit-learn

Why Array API matters for scikit-learn

- scikit-learn grew around **NumPy-style arrays on the CPU**

- many workflows now use **GPUs** and **other array libraries**

- goal:
  let serious work **stay on the device and library** the data already lives in,
  instead of silently pulling everything back to CPU and NumPy

Why Array API matters for scikit-learn

- scikit-learn is an array consuming library:
  inputs are arrays; outputs are arrays

- support is **experimental** and has grown since **1.3** (June 2023):
  more estimators and functions over time

- users can bring **CuPy** or **PyTorch** data today; **JAX** is being prepared

The Array API Standard

- **Consortium for Python Data API Standards** maintains the **Array API** standard (specification published at data-apis.org)

- **array-api-tests**: shared conformance test suite
- **array-api-strict**: strict, minimal implementation used heavily in scikit-learn CI
- **array-api-compat**: bridges NumPy, PyTorch, and CuPy where their APIs differ fromArray API standard; vendored in scikit-learn since 1.7
- **array-api-extra**: helpers for operations the core standard does not define; vendored in scikit-learn since 1.7

Using Array API compatible inputs in scikit-learn

- opt-in:
  - experimental in **SciPy**: **SCIPY_ARRAY_API=1** environment variable
  - scikit-learn global configuration (**set_config**) or the **config_context** context manager: **array_api_dispatch=True**
- scikit-learn "**Array API support (experimental)**" user docs

scikit learn

12.2.2. Example usage

The example code snippet below demonstrates how to use CuPy to run `LinearDiscriminantAnalysis` on a GPU:

```
>>> from sklearn.datasets import make_classification
>>> from sklearn import config_context
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
>>> import cupy

>>> X_np, y_np = make_classification(random_state=0)
>>> X_cu = cupy.asarray(X_np)
>>> y_cu = cupy.asarray(y_np)
>>> X_cu.device
<CUDA Device 0>

>>> with config_context(array_api_dispatch=True):
...     lda = LinearDiscriminantAnalysis()
...     X_trans = lda.fit_transform(X_cu, y_cu)
>>> X_trans.device
<CUDA Device 0>
```

GOSIM

GOSIM Paris 2026

02

Implementation and Challenges

Implementation in scikit-learn

Code | Issues 1.5k | Pull requests 528 | Agents | Discussions | Actions | Projects | Wiki | Security

Path for Adopting the Array API spec #22352

Closed

thomasjpfan opened on Feb 1, 2022 · edited by thomasjpfan Edits Member

I have been experimenting with adopting the Array API spec into scikit-learn. The Array API is one way for scikit-learn to run on other hardware such as GPUs.

I have some POCs on my fork for LinearDiscriminantAnalysis and GaussianMixture. Overall, there is runtime performance benefit when running on CuPy compared to NumPy, as shown in notesbooks for LDA (14x improvement) and GMM (7x improvement).

Official acceptance of the Array API in numpy is tracked as NEP 47.

Proposed User API

Here is the proposed API for dispatching. We require the array to adopt the Array API standard and we have a configuration option to turn on Array API dispatching:

Implementation in scikit-learn

Computations stay on the same **namespace** and **device** as the input arrays:

```
from sklearn.utils._array_api import get_namespace, device

xp, is_array_api = get_namespace(X)  # same library as `X` (CuPy, torch, numpy, strict, …)
device = device(X)                   # same device as `X`
…

# inside estimators:
# keep operations on namespace `xp` and
# create new arrays on namespace `xp` and `device`
```

Implementation in scikit-learn

- scikit-learn uses **array-api-strict** for testing and vendors both **array-api-compat** and **array-api-extra** since 1.7
- Array API standard is deliberately a subset of NumPy's surface
- add helpers in scikit-learn instead of assuming every backend exposes the same NumPy name

```
def _nanmin(X, axis=None, xp=None):
    # TODO: refactor once nan-aware reductions are standardized:
    # https://github.com/data-apis/array-api/issues/621
    xp, _, device_ = get_namespace_and_device(X, xp=xp)
    if _is_numpy_namespace(xp):
        return xp.asarray(numpy.nanmin(X, axis=axis))

    else:
        mask = xp.isnan(X)
        X = xp.min(
            xp.where(mask, xp.asarray(+xp.inf, dtype=X.dtype, device=device_), X),
            axis=axis,
        )
        # Replace Infs from all NaN slices with NaN again
        mask = xp.all(mask, axis=axis)
        if xp.any(mask):
            X = xp.where(mask, xp.asarray(xp.nan, dtype=X.dtype, device=device_), X)
        return X
```

GOSIM

GOSIM Paris 2026

Thanks!

:probabl.